

[DHS/2025/Q2]

2 You created a new online game called the Guessing Pair Game where a human player (client program) plays against the computer player (server program). At the beginning of the game, a board of 2 rows & 4 columns is generated and the 8 cells are populated randomly with 4 pairs of numbers range between 1 and 9 (inclusive). The aim of the game is to find the same number on the board by choosing 2 cells in each turn. When a pair of numbers is found, points are added to the human/computer player based on the number revealed. The human player will always start first followed by the Computer. The game ends when all pairs have been found.

Study the sample program output in the next three pages to determine your code design, output format and socket protocol. User inputs are underlined and comments are shown after #. [Note: there is no need to display the comments in your programs]

Sample server program (Computer player):

```
-----
GUESSING PAIR GAME
-----

Waiting for player to join
-----
Revealed Board: [[3, 2, 5, 5], [3, 7, 7, 2]]
#Revealed Board is only shown in Server Program

Board:
X | X | X | X
X | X | X | X

Player chose: row 1: 0, col 1: 1, row 2: 1, col 2: 3
Player Score: 2
Computer Score: 0

Board:
X | 2 | X | X
X | X | X | 2

Computer chose: row 1: 1, col 1: 1, row 2: 1, col 2: 2
Player Score: 2
Computer Score: 7

Board:
X | 2 | X | X
X | 7 | 7 | 2

Player chose: row 1: 0, col 1: 2, row 2: 0, col 2: 3
Player Score: 7
Computer Score: 7

Board:
X | 2 | 5 | 5
X | 7 | 7 | 2

Computer chose: row 1: 1, col 1: 0, row 2: 0, col 2: 0
Player Score: 7
Computer Score: 10
```

[DHS/2025/Q2 Cont']

```
Board:
 3 | 2 | 5 | 5
 3 | 7 | 7 | 2

Player lose!
-----
GAME ENDED
-----
```

```
Sample client program (Human player):
-----
GUESSING PAIR GAME
-----
Board:
 X | X | X | X
 X | X | X | X

INPUT first coordinates row number: 0
INPUT first coordinates column number: 1

INPUT second coordinates row number: 1
INPUT second coordinates column number: 3

Its a match! You scored 2 points
Player Score: 2
Computer Score: 0

Board:
 X | 2 | X | X
 X | X | X | 2

Computer chose: row 1: 1, col 1: 1, row 2: 1, col 2: 2
Its a match! Computer scored 7 points
Player Score: 2
Computer Score: 7

Board:
 X | 2 | X | X
 X | 7 | 7 | 2

INPUT first coordinates row number: 0 #cell already revealed so
INPUT first coordinates column number: 1 #prompt for another cell
INPUT first coordinates row number: 0
INPUT first coordinates column number: 2

INPUT second coordinates row number: 5 #user input is
INPUT second coordinates column number: 7 #out of range
INPUT second coordinates row number: 0 #second input is the same
INPUT second coordinates column number: 2 #as the first input
INPUT second coordinates row number: 0
INPUT second coordinates column number: 3

Its a match! You scored 5 points
Player Score: 7
Computer Score: 7
```

[DHS/2025/Q2 Cont']

```
Board:
X | 2 | 5 | 5
X | 7 | 7 | 2

Computer chose: row 1: 1, col 1: 0, row 2: 0, col 2: 0
Its a match! Computer scored 3 points
Player Score: 7
Computer Score: 10

Board:
3 | 2 | 5 | 5
3 | 7 | 7 | 2

You lose!
-----
GAME ENDED
-----
```

Task 2.1

The Guessing Pair Game is implemented using Object-Oriented Programming.

The class `GuessingPairGame` contains four properties:

- `board` stores a 2D array (2 rows of 4 columns) with each cell containing a number
- `revealed` stores a 2D array (2 rows of 4 columns) tracking the visibility status of each corresponding cell in `board` (`True` indicates revealed, `False` indicates hidden)
- `player_score` stores the human player's score
- `computer_score` stores the computer's score

The class `GuessingPairGame` contains the following methods:

- `create_board()` generates the initial board which is a 2D array (2 rows of 4 columns) with cell values set to `-1`
- `create_revealed_board()` generates the initial revealed board which is a 2D array (2 rows of 4 columns) with cell values set to `False`
- `populate_board()` fills the board with 4 pairs of random numbers that range between 1 and 9 (inclusive). The placement of the numbers is also randomised
- `set_board(values)` fills the board according to the provided values string (example values string: `7,9,5,7,9,1,5,1`)
- `print_board()` displays the board. Shows `x` for cells that remain hidden and shows the actual value for revealed cells
- `board_in_string()` converts the board values into a string format, e.g. `7,9,5,7,9,1,5,1`
- `valid_choice(row, column)` returns `True` if the selected row and column are within bounds and the chosen cell is not yet revealed, otherwise returns `False`
- `play_move(row1, column1, row2, column2)` determines if the two selected cells contain the same number, and if they match, update their revealed status in the `revealed` board to `True` and returns the score equivalent to the number revealed. Returns `0` if the two cells do not match
- `is_game_over()` checks if all cells have been revealed. If all are revealed, returns `True`, otherwise returns `False`

[DHS/2025/Q2 Cont']

You are provided with the template program for Guessing Pair Game, `GuessingPairGame.py`. Some of the codes have been provided for you. Complete the code for `populate_board()`, `play_move(row1, column1, row2, column2)` and `is_game_over()`.

[Note: DO NOT rename the file `GuessingPairGame.py`. Write your name as a comment on the first line of code in `GuessingPairGame.py`. [9]

Task 2.2

Using socket programming, complete the client program to:

1. establish connection to the server program
2. receive the board values in string format from the server program
3. load the received board values into its own `GuessingPairGame` instance using the relevant method
4. collect user input for the 2 coordinates. Request user to re-enter coordinates if the provided coordinates are invalid. Validate using the relevant method and conditions.
5. modify the `revealed` board and `player_score` using the relevant method or otherwise according to user input and show an appropriate message.
6. send the user selected coordinates in the appropriate format to the server program to modify its `revealed` board and `player_score`
7. verify if the game has ended using the relevant method
8. receive the 2 coordinates from the server program (Computer player's move)
9. update the `revealed` board and `computer_score` using the relevant method or otherwise according to computer player's selected coordinates and show an appropriate message.
10. verify if the game has ended using the relevant method
11. repeat Steps 4 to 10 until the game has ended
12. display whether Player wins or loses according to the score

You are provided with the completed server program and client template program, `SERVER.py` and `CLIENT.py` respectively. Complete the client program and rename as `CLIENT_<your name>_<centre number>_<index number>.py` [14]

Save all your files for Task 2.